

Diabetes

jae

2026-03-14

##Installing package This section loads the packages required for data import, cleaning, visualization, modeling, and evaluation. The package installation lines are kept commented out because they are only needed the first time the analysis is run on a new machine.

###looking at only one country file before worjing on the data Before combining all country files, one country file was inspected to understand the file structure, including the placement of the country name, year labels, indicator names, and values. This step helped guide the cleaning function used for all files.

```
df_raw <- read_excel("/Users/JOHN/Desktop/Trial data mining work/Algeria Country Table Export.xlsx", col
```

```
## New names:
## * ' -> '...1'
## * ' -> '...2'
## * ' -> '...3'
## * ' -> '...4'
## * ' -> '...5'
```

```
country_name <- df_raw %>%
  slice(1) %>%      # take row 1
  select(1) %>%     # take column 1
  unlist() %>%
  as.character()

years <- df_raw %>%
  slice(2) %>%      # take row 2
  select(2:5) %>%   # take columns 2 to 5
  unlist() %>%
  as.numeric()

df_clean <- df_raw %>%
  slice(-c(1:3)) %>%      # remove first 3 rows (country + years + blank)
  filter(!is.na(`...1`)) %>%      # keep only rows where column 1 is not blank
  filter(!str_detect(`...1`, "Diabetes|IFG|IGT|Mortality|Hyperglycaemia|Type1|Demographics"))

colnames(df_clean) <- c("Indicator", years)

df_clean <- df_clean %>%
  mutate(across(-Indicator, ~na_if(as.character(.), "-"))) %>% # change "-" into NA
  mutate(across(-Indicator, as.numeric))
```

```

df_long <- df_clean %>%
  pivot_longer(
    cols = -Indicator,
    names_to = "Year",
    values_to = "Value"
  ) %>%
  mutate(
    Country = country_name,
    Year = as.numeric(Year)
  )

df_wide <- df_long %>%
  pivot_wider(
    names_from = Indicator,
    values_from = Value
  )

```

File Listing all files and Cleaning Function This section identifies all Excel files in the project folder and defines a reusable function for cleaning each country file. The function extracts the country name, retrieves the year labels, removes section headers, converts values to numeric format, and reshapes the file into long format. You can also embed plots, for example:

```

file_list <- list.files(
  path = "/Users/JOHN/Desktop/Trial data mining work/african data idf",
  pattern = "\\..xlsx$",
  full.names = TRUE
)

#Function to clean one country file
clean_country_file <- function(file_path) {

  df_raw <- read_excel(file_path, col_names = FALSE)
  colnames(df_raw) <- paste0("V", 1:ncol(df_raw))

  country_name <- as.character(df_raw[1,1])
  years <- as.numeric(unlist(df_raw[2, 2:5]))

  df_clean <- df_raw %>%
    slice(-c(1:3)) %>%
    filter(!is.na(V1)) %>%
    filter(
      !str_trim(V1) %in% c(
        "Diabetes",
        "IFG",
        "IGT",
        "Mortality",
        "Hyperglycaemia in pregnancy",
        "Type 1 diabetes",
        "Demographics"
      )
    )

  colnames(df_clean) <- c("Indicator", years)
}

```

##Combine all countries This section applies the cleaning function to all country files, combines them into one long dataset, filters the analysis to the 2024 reference year, reshapes the data to wide format, and selects the variables required for modeling.

[illegible]

```
names(master_long)
```

```
# Filter to 2024 and reshape
master_data <- master_long %>%
  filter(Year == 2024) %>%
  pivot_wider(
    names_from = Indicator,
    values_from = Value
  ) %>%
  select(
    Country,
    Prevalence_percent = `Age-standardised prevalence of diabetes, %`,
    Undiagnosed_percent = `Proportion of people with undiagnosed diabetes, %`,
    Diabetes_Deaths = `Deaths attributable to diabetes`,
    Death_Proportion_percent = `Proportion of diabetes-related deaths in people 20-79 y, %`,
    Health_Expenditure_USD = `Total diabetes-related health expenditure, USD million`,
    Adult_Population_1000s = `Total adult population (20-79 y), in 1,000s`
  ) %>%
  mutate(across(-Country, as.numeric)) %>%
  na.omit()
nrow(master_data)
```

```
## [1] 48
```

```
length(unique(master_data$Country))
```

```
## [1] 48
```

##Data Checks and Outcome Variable Construction This section checks the cleaned dataset, calculates the median prevalence threshold, and creates the binary outcome variable used in the classification models. Countries are classified as either high prevalence or low prevalence based on the sample median of age-standardised diabetes prevalence

```
master_data %>%  
  count(Country) %>%  
  arrange(desc(n))
```

```
## # A tibble: 48 x 2  
##   Country      n  
##   <chr>      <int>  
## 1 Algeria      1  
## 2 Angola      1  
## 3 Benin       1  
## 4 Botswana    1  
## 5 Burkina Faso 1  
## 6 Burundi     1  
## 7 Cabo Verde  1  
## 8 Cameroon    1  
## 9 Central African Republic 1  
## 10 Chad       1  
## # i 38 more rows
```

```
#####checking number of rows and column  
dim(master_data)
```

```
## [1] 48 7
```

```
##it tells you the distinct countries present in your stacked dataset.  
unique(master_long$Country)
```

```
## [1] "Algeria"      "Angola"  
## [3] "Benin"       "Botswana"  
## [5] "Burkina Faso" "Burundi"  
## [7] "Cabo Verde"   "Cameroon"  
## [9] "Central African Republic" "Chad"  
## [11] "Comoros"      "Congo"  
## [13] "Côte d'Ivoire" "Democratic Republic of the Congo"  
## [15] "Djibouti"     "Egypt"  
## [17] "Equatorial Guinea" "Eritrea"  
## [19] "Eswatini"     "Ethiopia"  
## [21] "Gabon"        "Gambia"  
## [23] "Ghana"        "Guinea-Bissau"  
## [25] "Guinea"       "Kenya"
```

```
## [27] "Lebanon"           "Liberia"
## [29] "Libya"             "Madagascar"
## [31] "Malawi"            "Mali"
## [33] "Mauritania"        "Mauritius"
## [35] "Mayotte"           "Morocco"
## [37] "Mozambique"         "Namibia"
## [39] "Niger"              "Nigeria"
## [41] "Rwanda"             "Sao Tome and Principe"
## [43] "Senegal"            "Seychelles"
## [45] "Sierra Leone"      "Somalia"
## [47] "South Sudan"        "Sudan"
## [49] "Togo"               "Tunisia"
## [51] "Uganda"             "Zambia"
## [53] "Zimbabwe"
```

```
##checking for misssing
colSums(is.na(master_data))
```

```
##           Country      Prevalence_percent      Undiagnosed_percent
##                0                0                0
##      Diabetes_Deaths Death_Proportion_percent Health_Expenditure_USD
##                0                0                0
##      Adult_Population_1000s
##                0
```

```
sum(is.na(master_data))
```

```
## [1] 0
```

```
##Which specific countries are missing data?
master_data %>% filter(if_any(everything(), is.na)) %>% select(Country)
```

```
## # A tibble: 0 x 1
## # i 1 variable: Country <chr>
```

```
###Median prevalence
median_prev <- median(master_data$Prevalence_percent, na.rm = TRUE)

##Create binary outcome
master_data <- master_data %>%
  mutate(
    High_Prevalence = ifelse(Prevalence_percent >= median_prev, "High", "Low"),
    High_Prevalence = factor(High_Prevalence, levels = c("Low", "High"))
  )
##List high-prevalence countries
high_prev_countries <- master_data %>%
  filter(High_Prevalence == "High") %>%
  select(Country, Prevalence_percent) %>%
  arrange(desc(Prevalence_percent))
high_prev_countries
```

```
## # A tibble: 24 x 2
##   Country          Prevalence_percent
##   <chr>              <dbl>
## 1 Egypt              22.4
## 2 Mauritius           20.1
## 3 Sudan               19
## 4 Algeria            17.5
## 5 Tunisia            16
## 6 Lebanon            12.3
## 7 Sao Tome and Principe 12.1
## 8 Morocco            11.9
## 9 Comoros            10.8
## 10 Zambia             10.3
## # i 14 more rows
```

##Descriptive Statistics and Exploratory Data Analysis This section summarizes the diabetes-related indicators across the sampled African countries, checks the class balance, and visualizes the distribution of prevalence using a histogram. These outputs support the descriptive findings reported in the main paper.

```
desc_stats <- master_data %>%
  summarise(
    Mean_Prevalence = mean(Prevalence_percent, na.rm = TRUE),
    SD_Prevalence = sd(Prevalence_percent, na.rm = TRUE),
    Min_Prevalence = min(Prevalence_percent, na.rm = TRUE),
    Median_Prevalence = median(Prevalence_percent, na.rm = TRUE),
    Max_Prevalence = max(Prevalence_percent, na.rm = TRUE),

    Mean_Undiagnosed = mean(Undiagnosed_percent, na.rm = TRUE),
    Mean_Deaths = mean(Diabetes_Deaths, na.rm = TRUE),
    Mean_Death_Prop = mean(Death_Proportion_percent, na.rm = TRUE),
    Mean_Health_Exp = mean(Health_Expenditure_USD, na.rm = TRUE),
    Mean_Adult_Pop = mean(Adult_Population_1000s, na.rm = TRUE)
  )
desc_stats
```

```
## # A tibble: 1 x 10
##   Mean_Prevalence SD_Prevalence Min_Prevalence Median_Prevalence Max_Prevalence
##   <dbl>          <dbl>          <dbl>          <dbl>          <dbl>
## 1          7.38          4.84          1.5          5.4          22.4
## # i 5 more variables: Mean_Undiagnosed <dbl>, Mean_Deaths <dbl>,
## #   Mean_Death_Prop <dbl>, Mean_Health_Exp <dbl>, Mean_Adult_Pop <dbl>
```

```
t(desc_stats)
```

```
##           [,1]
## Mean_Prevalence  7.377083
## SD_Prevalence    4.841190
## Min_Prevalence   1.500000
## Median_Prevalence 5.400000
## Max_Prevalence   22.400000
## Mean_Undiagnosed 66.056250
## Mean_Deaths      6421.506250
## Mean_Death_Prop   5.747917
```

```
## Mean_Health_Exp      299.691667
## Mean_Adult_Pop      13542.785417
```

```
#####Check class balance for the target variable
table(master_data$High_Prevalence)
```

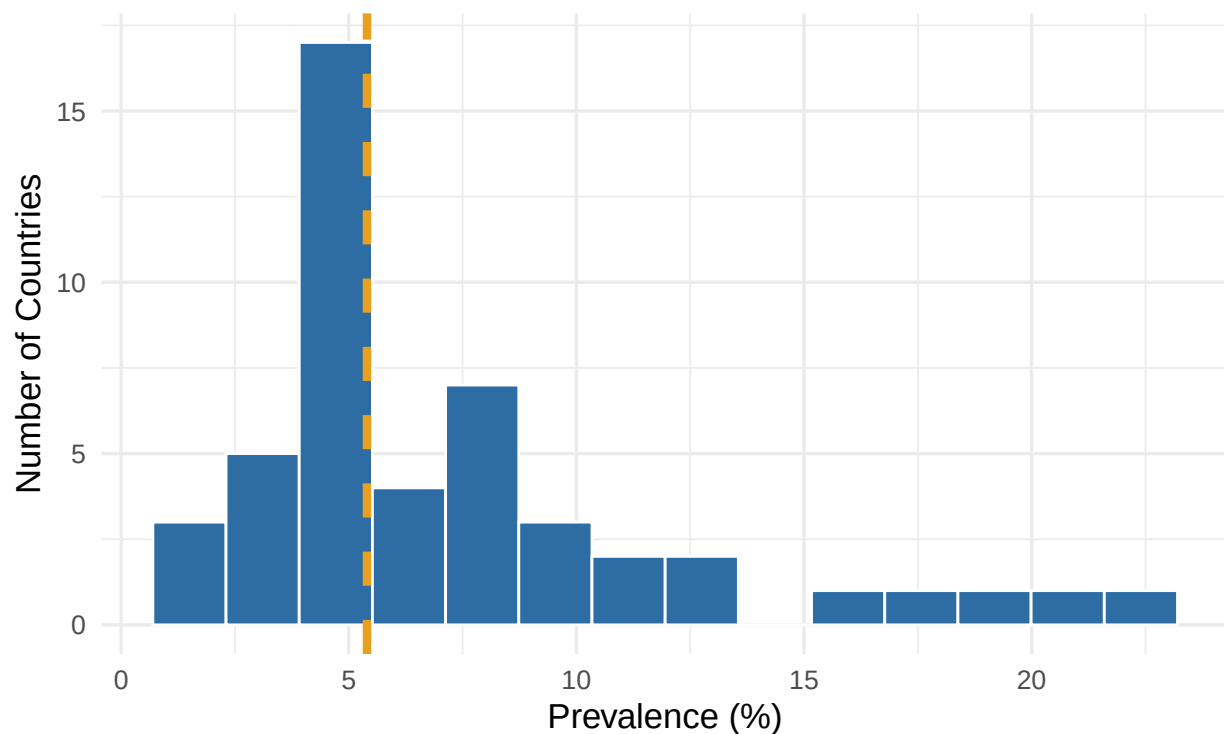
```
##
## Low High
##  24   24
```

```
####showing the balance in % (percentages)
prop.table(table(master_data$High_Prevalence)) * 100
```

```
##
## Low High
##  50   50
```

```
###Histogram
p1 <- ggplot(master_data, aes(x = Prevalence_percent)) +
  geom_histogram(bins = 14, fill = "#2E6DA4", colour = "white", linewidth = 0.5) +
  geom_vline(xintercept = median_prev, colour = "#E8A020",
            linewidth = 1.5, linetype = "dashed") +
  labs(
    title = "Distribution of Age-Standardised Diabetes Prevalence",
    subtitle = paste0("African Countries - IDF Atlas 2024 (N = ", nrow(master_data), ")"),
    x = "Prevalence (%)",
    y = "Number of Countries"
  ) +
  theme_minimal(base_size = 13)
print(p1)
```


Distribution of Age-Standardised Diabetes Prevalence African Countries — IDF Atlas 2024 (N = 48)



Model Dataset Preparation and Train-Test Split

This section prepares the final modeling dataset by excluding the country identifier and the prevalence variable used to define the outcome. The data are then split into training and testing sets to support model estimation and evaluation.

```
model_data <- master_data %>%  
  select(  
    High_Prevalence,  
    Undiagnosed_percent,  
    Diabetes_Deaths,  
    Death_Proportion_percent,  
    Health_Expenditure_USD,  
    Adult_Population_1000s  
  )  
  
##for reproducibility  
set.seed(123)  
  
train_index <- createDataPartition(model_data$High_Prevalence, p = 0.7, list = FALSE)  
  
train_set <- model_data[train_index, ]  
test_set <- model_data[-train_index, ]
```

CART Model Estimation and Pruning

This section fits the CART decision tree model, examines the complexity parameter table, selects the optimal pruning parameter, and produces the final pruned tree for interpretation.

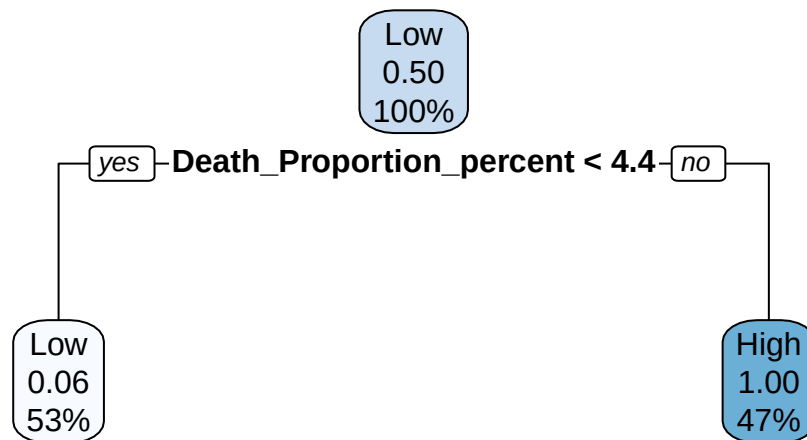
```
cart_tree <- rpart(  
  High_Prevalence ~ .,  
  data = train_set,  
  method = "class",  
  control = rpart.control(cp = 0.001, minsplit = 5, xval = 10)  
)  
  
print(cart_tree)
```

```
## n= 34  
##  
## node), split, n, loss, yval, (yprob)  
##      * denotes terminal node  
##  
## 1) root 34 17 Low (0.50000000 0.50000000)  
##    2) Death_Proportion_percent< 4.35 18 1 Low (0.94444444 0.05555556) *  
##    3) Death_Proportion_percent>=4.35 16 0 High (0.00000000 1.00000000) *
```

```
printcp(cart_tree)
```

```
##  
## Classification tree:  
## rpart(formula = High_Prevalence ~ ., data = train_set, method = "class",  
##       control = rpart.control(cp = 0.001, minsplit = 5, xval = 10))  
##  
## Variables actually used in tree construction:  
## [1] Death_Proportion_percent  
##  
## Root node error: 17/34 = 0.5  
##  
## n= 34  
##  
##      CP nsplit rel error  xerror    xstd  
## 1 0.94118      0 1.000000 1.35294 0.160462  
## 2 0.00100      1 0.058824 0.11765 0.080705
```

```
### finding cp=complexity parameter(controls the rpruning of the tree)  
best_cp <- cart_tree$cptable[which.min(cart_tree$cptable[, "xerror"]), "CP"]  
####Pruning the tree using cp  
pruned_cart_tree <- prune(cart_tree, cp = best_cp)  
  
##plot the pruned tree  
### other options for colours greens,reds,purples,oranges,grays  
rpart.plot(pruned_cart_tree, box.palette = "Blues")
```



CART Model Evaluation

This section evaluates the pruned CART model using predictions on the test set. A confusion matrix is generated, followed by extraction of the four key evaluation metrics: accuracy, precision, recall, and F1-score.

```

cart_pred <- predict(pruned_cart_tree, test_set, type = "class")
####confusion matrix
cart_cm <- confusionMatrix(cart_pred, test_set$High_Prevalence, positive = "High")
cart_cm

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction Low High
##           Low    6    0
##           High    1    7
##
##           Accuracy : 0.9286
##           95% CI : (0.6613, 0.9982)
##           No Information Rate : 0.5
##           P-Value [Acc > NIR] : 0.0009155
##
##           Kappa : 0.8571

```

```
##
## McNemar's Test P-Value : 1.0000000
##
##      Sensitivity : 1.0000
##      Specificity : 0.8571
##      Pos Pred Value : 0.8750
##      Neg Pred Value : 1.0000
##      Prevalence : 0.5000
##      Detection Rate : 0.5000
##      Detection Prevalence : 0.5714
##      Balanced Accuracy : 0.9286
##
##      'Positive' Class : High
##
```

```
##big four matrix
cart_accuracy <- cart_cm$overall["Accuracy"]
cart_precision <- cart_cm$byClass["Pos Pred Value"]
cart_recall <- cart_cm$byClass["Sensitivity"]
cart_f1 <- 2 * ((cart_precision * cart_recall) / (cart_precision + cart_recall))

cart_accuracy
```

```
## Accuracy
## 0.9285714
```

```
cart_precision
```

```
## Pos Pred Value
##      0.875
```

```
cart_recall
```

```
## Sensitivity
##      1
```

```
cart_f1
```

```
## Pos Pred Value
##      0.9333333
```

Logistic Regression Benchmark Model

This section fits logistic regression as a benchmark classifier using the same predictors and outcome. The binary factor outcome is recoded to numeric format for estimation, and the resulting model is evaluated on the test set.

```
train_glm <- train_set %>%
  mutate(High_Prevalence_num = ifelse(High_Prevalence == "High", 1, 0)) %>%
  select(-High_Prevalence)
```

```

test_glm <- test_set %>%
  mutate(High_Prevalence_num = ifelse(High_Prevalence == "High", 1, 0)) %>%
  select(-High_Prevalence)
##Building the model using our target and all other variable
logit_model <- glm(
  High_Prevalence_num ~ .,
  data = train_glm,
  family = binomial()
)

```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```

##describing the data
summary(logit_model)

```

```

##
## Call:
## glm(formula = High_Prevalence_num ~ ., family = binomial(), data = train_glm)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -6.547e+02  1.266e+06  -0.001    1.000
## Undiagnosed_percent  4.349e+00  1.461e+04   0.000    1.000
## Diabetes_Deaths    -1.857e-03  1.786e+01   0.000    1.000
## Death_Proportion_percent  8.371e+01  7.140e+04   0.001    0.999
## Health_Expenditure_USD  -2.372e-01  4.079e+02  -0.001    1.000
## Adult_Population_1000s  5.008e-04  1.718e+01   0.000    1.000
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 4.7134e+01  on 33  degrees of freedom
## Residual deviance: 3.6298e-09  on 28  degrees of freedom
## AIC: 12
##
## Number of Fisher Scoring iterations: 25

```

```
##predictionsabout the responses
```

```

logit_prob <- predict(logit_model, newdata = test_glm, type = "response")
##setting thresholds:If the model is more than 50% sure a country is "High," you label it "High." Other
logit_pred <- ifelse(logit_prob >= 0.5, "High", "Low")
logit_pred <- factor(logit_pred, levels = c("Low", "High"))
###confusion matrix
logit_cm <- confusionMatrix(logit_pred, test_set$High_Prevalence, positive = "High")
logit_cm

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction Low High

```

```
##      Low    7    0
##      High   0    7
##
##              Accuracy : 1
##              95% CI : (0.7684, 1)
##      No Information Rate : 0.5
##      P-Value [Acc > NIR] : 6.104e-05
##
##              Kappa : 1
##
##      McNemar's Test P-Value : NA
##
##              Sensitivity : 1.0
##              Specificity : 1.0
##      Pos Pred Value : 1.0
##      Neg Pred Value : 1.0
##              Prevalence : 0.5
##      Detection Rate : 0.5
##      Detection Prevalence : 0.5
##      Balanced Accuracy : 1.0
##
##      'Positive' Class : High
##
```

##big four matrix

```
logit_accuracy <- logit_cm$overall["Accuracy"]
logit_precision <- logit_cm$byClass["Pos Pred Value"]
logit_recall    <- logit_cm$byClass["Sensitivity"]
logit_f1        <- 2 * ((logit_precision * logit_recall) / (logit_precision + logit_recall))
```

```
logit_accuracy
```

```
## Accuracy
##      1
```

```
logit_precision
```

```
## Pos Pred Value
##      1
```

```
logit_recall
```

```
## Sensitivity
##      1
```

```
logit_f1
```

```
## Pos Pred Value
##      1
```

##Model Comparison and Variable Importance This final analytical section compares CART and logistic regression using the selected performance metrics and then extracts the variable importance values from the pruned CART model.

```
model_comparison <- data.frame(
  Model = c("CART", "Logistic Regression"),
  Accuracy = c(as.numeric(cart_accuracy), as.numeric(logit_accuracy)),
  Precision = c(as.numeric(cart_precision), as.numeric(logit_precision)),
  Recall = c(as.numeric(cart_recall), as.numeric(logit_recall)),
  F1_Score = c(as.numeric(cart_f1), as.numeric(logit_f1))
)

model_comparison
```

```
##           Model  Accuracy Precision Recall  F1_Score
## 1           CART 0.9285714    0.875     1 0.9333333
## 2 Logistic Regression 1.0000000    1.000     1 1.0000000
```

```
####inspecting tree to see variables that were more influential
pruned_cart_tree
```

```
## n= 34
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 34 17 Low (0.50000000 0.50000000)
##    2) Death_Proportion_percent< 4.35 18 1 Low (0.94444444 0.05555556) *
##    3) Death_Proportion_percent>=4.35 16 0 High (0.00000000 1.00000000) *
```

```
pruned_cart_tree$variable.importance
```

```
## Death_Proportion_percent      Undiagnosed_percent  Adult_Population_1000s
##           15.111111            8.500000            6.611111
##      Diabetes_Deaths    Health_Expenditure_USD
##           4.722222            2.833333
```

Outputs Used in the Main Report

This section gathers the specific outputs used in the written report, including the class distribution, descriptive statistics, saved histogram, CART results, saved tree figure, and variable importance values.

```
table(master_data$High_Prevalence)
```

```
##
##  Low High
##   24   24
```

```
prop.table(table(master_data$High_Prevalence)) * 100
```

```
##
##  Low High
##   50   50
```

##2.DESRIPTIVE STATISTICS

```
t(desc_stats)
```

```
##           [,1]
## Mean_Prevalence      7.377083
## SD_Prevalence        4.841190
## Min_Prevalence        1.500000
## Median_Prevalence     5.400000
## Max_Prevalence       22.400000
## Mean_Undiagnosed     66.056250
## Mean_Deaths          6421.506250
## Mean_Death_Prop       5.747917
## Mean_Health_Exp      299.691667
## Mean_Adult_Pop      13542.785417
```

##HISTOGRAM OF PREVALENCE

```
ggsave("figure1_histogram.png", plot = p1, width = 8, height = 5, dpi = 300)
```

##This will save the histogram as an image in your working folder.

##So you go to insert and that folder to insert in word

##GETTING THE CART RESULTS IN RSTUDIO

```
cart_cm ##FOR VIEWING ALL
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction Low High
```

```
##           Low      6      0
```

```
##           High     1      7
```

```
##
```

```
##           Accuracy : 0.9286
```

```
##           95% CI : (0.6613, 0.9982)
```

```
##           No Information Rate : 0.5
```

```
##           P-Value [Acc > NIR] : 0.0009155
```

```
##
```

```
##           Kappa : 0.8571
```

```
##
```

```
##           McNemar's Test P-Value : 1.0000000
```

```
##
```

```
##           Sensitivity : 1.0000
```

```
##           Specificity : 0.8571
```

```
##           Pos Pred Value : 0.8750
```

```
##           Neg Pred Value : 1.0000
```

```
##           Prevalence : 0.5000
```

```
##           Detection Rate : 0.5000
```

```
##           Detection Prevalence : 0.5714
```

```
##           Balanced Accuracy : 0.9286
```

```
##
```

```
##           'Positive' Class : High
```

```
##
```



```
###VIEWING THE MAIN PERFORMANCE VALUES OR KEY ONES
```

```
cart_accuracy
```

```
## Accuracy  
## 0.9285714
```

```
cart_precision
```

```
## Pos Pred Value  
## 0.875
```

```
cart_recall
```

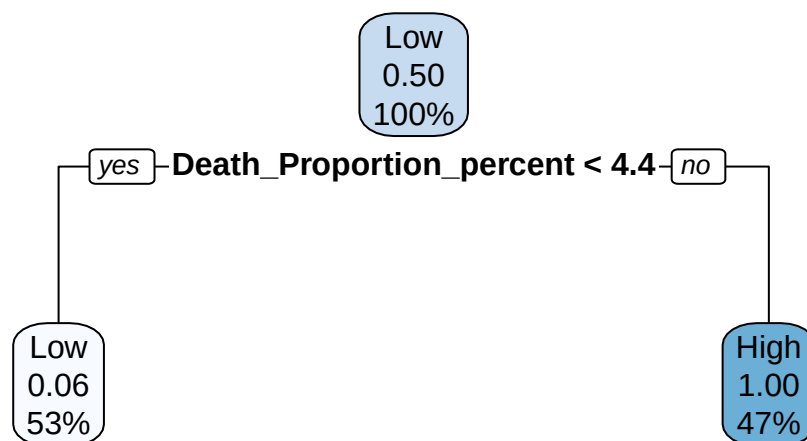
```
## Sensitivity  
## 1
```

```
cart_f1
```

```
## Pos Pred Value  
## 0.9333333
```

```
###PRUNED DECISION TREE
```

```
rpart.plot(pruned_cart_tree, box.palette = "Blues")
```



##SAVING IT IN THE FOLDER

```
png("figure2_pruned_tree.png", width = 1200, height = 800, res = 150)
rpart.plot(pruned_cart_tree, box.palette = "Blues")
dev.off()
```

```
## pdf
## 2
```

##GETTING VARIABLE IMPORTANCE DETAILS

```
pruned_cart_tree$variable.importance
```

##	Death_Proportion_percent	Undiagnosed_percent	Adult_Population_1000s
##	15.111111	8.500000	6.611111
##	Diabetes_Deaths	Health_Expenditure_USD	
##	4.722222	2.833333	